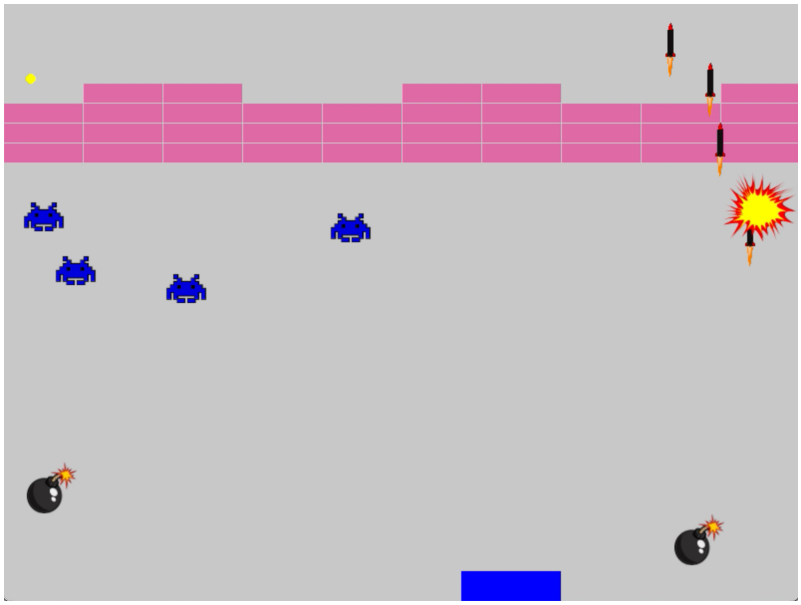
Applied Software Development Lab
Lab Handout 10: Now with Moving Pictures!

Learning objectives
- **Refactoring (cleaning up is always part of it)**
- **Catching exceptions**
- **Avoiding data congestion when reading the serial interface**
- **And above all: bitmaps and new game features**

Today's goal
- The controller should be connectable and usable even while the game is already running. But it should also be possible to remove it while the game is running. This is already supported in part – but only in part.
- If you turn the potentiometer quickly and use the push button at the same time, the game may "lag" (yes, that's really what people say!). The data you receive from the controller piles up. Even if you suddenly stop turning the potentiometer, the paddle keeps moving for a while.
- Of course, a game also needs sound, e.g., when the ball bounces off, when a spaceship is shot down, etc.
- A rocket should look like a rocket, a spaceship like a spaceship. In short: some things should be displayed with bitmaps and not just with geometric objects such as circles or rectangles.
- And then, of course, you need lots of new features, for example spaceships that can drop bombs, bricks that have to be hit several times, etc. And maybe you shouldn't have an unlimited supply of ammunition either.
- This handout uses a few examples to show how such new features can be implemented – so that afterwards you can also implement your own ideas on your own.

We will tackle these things in a seemingly arbitrary order. This is meant to show you what a typical process looks like in practice: at first, you only intend to add one improvement – and then one thing leads to another. You discover places in the code that should be reorganized so that new features fit in better. You find bugs that were hidden so far. And you get ideas for the next feature that you didn't even have on your radar.



Preparation: part 1

As usual, we work with two terminals: **left**: editing source code, **right**: compiling code, testing the game. Clone the repository with

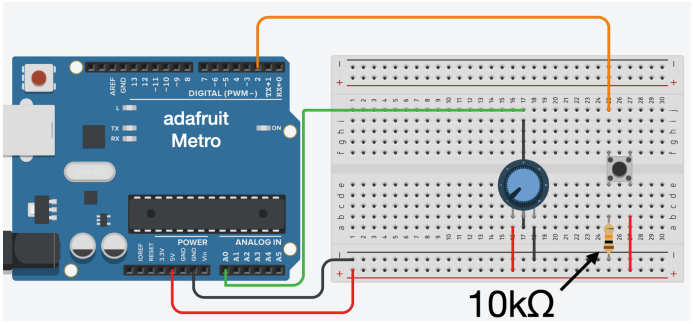
```
git clone https://gitlab.uni-ulm.de/USERNAME/asp-ulm
```

Then, in both terminals, change into the directory with the Python code:

```
cd asp-ulm/python
```

Preparation: part 2

You should build the familiar circuit with push button and potentiometer on the breadboard and connect it correctly to the microcontroller. The push button is connected to digital input pin 2, the potentiometer to analog input A0. On the right, you can see a picture of the complete setup.



Preparation: part 3

In the first step today, the rockets should no longer be drawn as simple filled circles but by an image. So that you don't have to draw the images yourselves, I have created a Git repository with additional material. **However, it is a bit tricky** to add these files to your own repository in the end. Here are the instructions:

1. Make sure that you are in the subdirectory of your project. In the terminal, enter `pwd`. This prints the current directory.
   On the lab computers, this should be `/home/asp/asp-ulm/python`.
2. Clone my repository with the additional material:

   ```
   git clone https://github.com/michael-lehn/asp-ulm-data.git
   ```

   This creates a new subdirectory `asp-ulm-data`.
   Confirm this by displaying the content of the current directory with `ls`.
3. Rename the folder to `data`: `mv asp-ulm-data data`
   Use `ls` again to display what is in the current directory, and confirm that the subfolder is now called `data`.
4. Now comes the delicate part, because something gets deleted. First use

   ```
   ls -a data
   ```

   to display everything that is in the folder. It should contain useful things such as image and sound files, but also a directory `.git`. And exactly this `.git` has to go! It contains administrative files because this directory was originally cloned from my repository. You don't want to drag this stuff along.
   To do so, run:

   ```
   rm -rf data/.git
   ```

   If you then run

   ```
   ls -a data
   ```

   again, `.git` should be gone.
5. Now add the folder with its entire content to your own repository:

   ```
   git add data/*
   git commit -a -m "images and sound files"
   git push
   ```

© Michael Lehn | CC BY-SA 4.0

Step 1: Finally, here we go – with images!

Connect the microcontroller and run `usb_check`.
Then open the file `ship.py` with the class `Ship`, which is responsible for the spaceships.
They should no longer be drawn as simple filled circles, but look like in Space Invaders.

In the constructor (i.e., the `__init__` method), add the following new line:

```
self.img = pygame.image.load("data/space_invader0.png")
```

This gives every Ship object a new attribute img, which is initialized with an image object. And the image object was created by loading the file `data/space_invader0.png`.

Now this image should be drawn instead of the circle. So change the method `draw` so that it contains only this line:

```
pygame.Surface.blit(Game.screen, self.img, (self.x, self.y))
```

The `blit` method copies an image (more precisely: a `Surface` object) to a specific position on another surface, here the screen. This lets you display arbitrary images instead of simple shapes.

💥 Fun fact: where does "BitBlt" come from?

The term **BitBlt** (bit block transfer) comes from the legendary Xerox PARC research labs of the 1970s. That is where the graphical user interface (GUI) as we know it today was developed – with windows, icons, and mouse pointers. To display all of this smoothly, a fast method was needed to copy image data from one memory location to another. That was the birth of the **BitBlt function** – and the word **blit** has survived to this day as a short form in many graphics libraries (such as Pygame).

Task

1. Test the new feature by running `python breakout.py`.
2. As mentioned above, `blit()` is a method – namely of the class `pygame.Surface`. Both `self.img` and `Game.screen` are objects of this class. Now change the line in the `draw` method to:

   ```
   Game.screen.blit(self.img, (self.x, self.y))
   ```

   This does **exactly the same** as before. Internally, Python more or less rewrites it into what was there before. In the new form, however, it is clearer what is meant: **The blit method is applied to the Game.screen object to copy the self.img object to the position (self.x, self.y).**

3. We still have a small cosmetic flaw: the images of the space invaders are currently drawn such that `(self.x, self.y)` is the top-left corner of the image.
   Concretely, this means, e.g., that rockets seem to "fly through" when you hit the image on its right side with a rocket. At the same time, the space invaders are already shot down when the rocket actually flies past them on the left.
   Test this! We don't want to solve problems that we don't know exactly.
4. To correct this, change the code in the `draw` method to:

   ```
   pos = self.img.get_rect()
   pos.center = (self.x, self.y)
   Game.screen.blit(self.img, pos)
   ```

   The `get_rect()` method returns an object of type `pygame.Rect`.
   Such objects represent rectangles (in this case, the border of the image) and can be centered by overwriting their center attribute.
   Test whether this solves the problem. If so, then:

   ```
   git commit -a -m "image for space invader"
   git push
   ```

© Michael Lehn | CC BY-SA 4.0

Step 2: More images

Open the file `rocket.py` with the class `Rocket`, which is responsible for the rockets. They should now, of course, look like real rockets!

Task

1. You have seen how it worked for the spaceships. Do the same for the rockets and use the image `data/rocket0.png` for it.
2. If you transferred the concept directly, the images of the rockets are currently aligned at their center.
   For rockets, this is actually not quite right – as with a soldering iron, it is the **tip** that is the dangerous part. Here, however, it is harder to test whether a space invader is only hit when it is (currently) really caught by the middle of the rocket. So let's briefly throttle the game speed a little so that such details can be seen better.
   In `breakout.py`, change the line:

   ```
   time.sleep(1/30)
   ```

   to:

   ```
   time.sleep(1)
   ```

   Then only one frame per second is drawn – so we see everything in slow motion. If you start the game now, you should immediately see that when a rocket is launched, half of it already sticks out of the paddle (our "launch pad").
   Now, in rocket.py, change the line:

   ```
   pos.center = (self.x, self.y)
   ```

   to:

   ```
   pos.midtop = (self.x, self.y)
   ```

   If you now restart the game, you will see that the rocket is aligned at its tip (the dangerous part).

3. Since you are currently running the game in slow motion, you may notice a problem that was hidden before: the controls are sluggish – people say they **lag** (*lag* = delay).
   If you quickly turn the potentiometer back and forth and then let go, the paddle still keeps moving back and forth for a while.
   This is a typical problem when data is produced and processed at different rates (in electronics, this is called *crossing clock domains*).
   We will take care of this in the next step.
   If everything else works (and you have observed the problem that we will solve in the next step), then:
   `git commit -a -m "rocket image"`
   `git push`

© Michael Lehn | CC BY-SA 4.0

Step 3: Processing data from the controller faster

There you will find the code that reads and processes data from the serial interface (via our module `controller`).
It should look roughly like this:

```
ser_data = controller.readdata()
if ser_data:
    if ser_data[0] == "A":
        rockets.append(Rocket(Paddle.x + Paddle.w // 2, Paddle.y, -10))
    elif ser_data[0] == "X":
        Paddle.set(ser_data[1] * 1.5)
```

The problem: This code is executed **only once per frame**, but the serial interface delivers data **much faster**.
As a result, data "piles up", as you saw in the last step.

Fortunately, in our case the problem can be solved very easily by changing the code to:

```
while True:
    ser_data = controller.readdata()
    if ser_data is None:
        break
    if ser_data[0] == "A":
        rockets.append(Rocket(Paddle.x + Paddle.w // 2, Paddle.y, -10))
    elif ser_data[0] == "X":
        Paddle.set(ser_data[1] * 1.5)
```

This way, the data is processed practically immediately: the loop reads data until no new data is available and processes it right away – e.g., by creating new rockets or setting the paddle position.

Task:

1. Check that with this change the game is **still very sluggish** – but only because we still draw **only one frame per second**.
   The delays caused by the data processing, however, are gone.
   If you now quickly turn the potentiometer and then stop, the paddle is drawn **immediately at the new position** in the next frame and then stays there.
2. Of course, we now want to play with at least 30 frames per second again.
   But this time we do it properly and no longer use `time.sleep()`, but the Clock that Pygame provides for this purpose.
   It is used according to this pattern:

   ```
   clock = pygame.time.Clock()
   while running:
       # here comes the code to handle events
       # and to draw a frame

       # and at the end of the loop:
       clock.tick(30)
   ```

   Apply this pattern and check that the game now runs smoothly again – possibly even more smoothly than before.

© Michael Lehn | CC BY-SA 4.0

What is behind this?

- Before the main loop, a Clock object is created. This object internally stores the current time.
- When you call the method tick(30), it checks:
  - **If less than 1/30 of a second has passed since the stored time**, it blocks briefly until the time span is over, and then stores the new current time.
  - **If enough time has already passed**, it simply stores the current time and continues immediately.

What is the advantage?

Drawing does not always take exactly the same amount of time!
How long it takes depends, e.g., on how many space invaders or balls are currently in the game, or on whether an object has just been hit.
As a rule, however, drawing takes considerably **less** than 1/30 of a second.
This mechanism ensures that a frame is always redrawn **after the same time span** – regardless of how much is going on.

3. If everything works, into the repository with it:
   `git commit -a -m "fixed lag and used clock"`
   `git push`

Step 4: Explosions 💥

After the boring data processing, we need some action – the space invaders should explode when they are hit!
As luck would have it, `data/explosion.png` already provides a suitable image for this.

The first approach is to check in the `draw()` method of the Ship class whether the attribute `alive` has the value `True` or `False`.
If it is `True`, the "normal" image is displayed – otherwise the one with the explosion.
Open the file `ship.py` and add the following line in the constructor (i.e., the `__init__` method):

```
self.die = pygame.image.load("data/explosion.png")
```

Then change the `draw` method as follows:

```
def draw(self):
    if self.alive:
        img = self.img
    else:
        img = self.die
    pos = img.get_rect()
    pos.center = (self.x, self.y)
    Game.screen.blit(img, pos)
```

Task

1. Make the change and test whether it works. You should notice that the program runs **without errors**, but **no explosions are displayed**.
2. **Why are no explosions displayed?**
   The problem is: a spaceship is no longer drawn at all when its `alive` attribute is `False`.

© Michael Lehn | CC BY-SA 4.0

Have a look at the file `breakout.py` – in the main loop, there is a line like this one:

```
ships = [item for item in ships if item.alive]
```

Here, `ships` is the list of spaceships that are drawn. When a ship has been hit, it is removed from the list **immediately** – and therefore no longer appears for a last "explosion frame".

This is how we solve the problem:

(a) Marking hits differently

In `breakout.py`, find the place where a hit has so far been handled with:

```
s.alive = False
```

Change it to:

```
s.hit = True
```

Tip: Test right away whether the program still runs without errors – that way you catch typos early. But don't be surprised: for now, the space invaders are invulnerable – we are not done yet!

(b) Initializing and using the `hit` attribute

In `ship.py`, initialize the new attribute in the constructor:

```
self.hit = False
```

In the `draw` method, change the code as follows:

```
def draw(self):
    if self.hit:
        img = self.die
        self.alive = False
    else:
        img = self.img
    pos = img.get_rect()
    pos.center = (self.x, self.y)
    Game.screen.blit(img, pos)
```

Test whether explosions are displayed now!
A space invader that has been hit should remain visible as an explosion for **exactly one frame** before it disappears from the list (and from life).

3. Now that a solution has been spoon-fed to you, you may solve something on your own again! 😄
When a rocket hits a space invader, **the rocket should explode, too**.
When a ball hits the space invader, the ball should **not** explode.

A possible approach (analogous to the previous subtask):

(a) In `breakout.py`, the attribute hit is set not only for the ship that was hit, **but also for the object that caused the hit**.

(b) In the `Ball` class, the `hit` attribute is ignored.
In the `Rocket` class, it is **handled the same way as in the Ship class**.

4. If everything works, into the repository with it:
`git commit -a -m "explosions"`
`git push`

© Michael Lehn | CC BY-SA 4.0

Step 5: Cleaning up the code

Have a look at the file `wall.py`. Currently, it is rather ugly that things like the width and height of the bricks and also the maximum number of bricks per row and column appear **hard-coded** as constants. You probably feel the same way I did: I had to look at it for a while to understand what exactly was specified. So I have decoded it for you:

- A brick is **80 pixels wide** and **20 pixels high**.
- There are at most **10 brick columns** and **30 brick rows**.

Also have a look at the file `game.py` in the other terminal.
There, the size of the drawing area is also hard-coded in the `init` method:
800 pixels wide, **600 pixels high**.
Fortunately, this fits together as it is currently specified:
- 80 pixels × 10 columns → 800 pixels wide.
- 20 pixels × 30 rows → 600 pixels high.

Great, so everything is fine, right?
Well, yes. But what if you want to change the size of the bricks? Or want more bricks?
Or what if you – like me – still want to understand the code in a few weeks?
Let's organize it better! Change `game.py` as follows:

```
import pygame

class Game:
    width, height = None, None
    screen = None
    brick_rows, brick_cols = 30, 10
    brick_width, brick_height = 80, 20

    def init():
        Game.width = Game.brick_cols * Game.brick_width
        Game.height = Game.brick_rows * Game.brick_height
        pygame.init()
        Game.screen = pygame.display.set_mode((Game.width, Game.height))

    def quit():
        pygame.quit()
```

Now it is specified in one central place how large the bricks are and how many rows and columns there are at most.

Task

1. Now it's your turn to make use of the new `Game` attributes in `wall.py`:
   - Replace every occurrence of `30` with `Game.brick_rows`.
   - Replace every occurrence of `10` with `Game.brick_cols`.
   - Replace every occurrence of `80` with `Game.brick_width`.
   - Replace every occurrence of `20` with `Game.brick_height`.

   Then test whether everything still works!
2. Also check whether you can change the new constants in `game.py` and everything still works. For example, change `Game.brick_cols` to `15`. Test whether the game still works then.
3. Afterwards, undo the change – a drawing area of 800 × 600 pixels is not bad if you want to play the game on a Raspberry Pi later on. 😄
4. If everything works:
   ```
   git commit -a -m "brick constants"
   git push
   ```

© Michael Lehn | CC BY-SA 4.0

Step 6: Even more code cleanup

Have a look at the file `breakout.py`.
The main loop is slowly getting quite large, because at the moment everything happens in it at once:
- We poll the serial interface for new data.
- We poll Pygame for new key events.
- We draw and move balls, spaceships, and rockets.
- We check whether there are collisions.

Now at least the **drawing and moving** of balls, spaceships, and rockets should be organized better.

Why?
(a) The code is almost identical.
(b) When the spaceships also drop bombs later, another list of objects will be added that have to be moved and drawn.

So: **Now is the right time to do this properly!**

And we'll do it completely right straight away:
The lists of the objects that are moved and drawn – i.e., `balls`, `rockets`, and `ships` – are moved into the Game class.
There will also be a method `draw()` there that manages all of these lists.

Change `game.py` as follows:

```
import pygame

class Game:
    width, height = None, None
    screen = None
    brick_rows = 30
    brick_cols = 15
    brick_width = 80
    brick_height = 20
    balls = []
    rockets = []
    ships = []

    def init():
        Game.width = Game.brick_cols * Game.brick_width
        Game.height = Game.brick_rows * Game.brick_height
        pygame.init()
        Game.screen = pygame.display.set_mode((Game.width, Game.height))

    def draw():
        for item in Game.balls + Game.rockets + Game.ships:
            item.draw()
            item.move()
        Game.balls = [item for item in Game.balls if item.alive]
        Game.rockets = [item for item in Game.rockets if item.alive]
        Game.ships = [item for item in Game.ships if item.alive]

    def quit():
        pygame.quit()
```

© Michael Lehn | CC BY-SA 4.0

Tasks

1. Adapt `breakout.py` so that the change takes effect:
   - Remove the code that has so far moved and drawn the balls, spaceships, and rockets. Instead, `Game.draw()` should now be called once.
   - All places where the lists `balls`, `rockets`, or `ships` are still accessed directly must now be adapted as well.
     There, write, e.g., `Game.balls` instead of `balls`, etc.
   - Then test whether everything works again!
2. To see that this cleanup is useful, too, make another small change in `wall.py`: whenever a brick is hit by a ball, a new space invader should enter the game. **Sounds complicated?** Don't worry, this **takes just two lines**:
   (a) At the beginning of the file, import the Ship class with `from ship import Ship`
   (b) When a brick has been hit, append a new Ship object to the list Game.ships. To do this, find the line with `Wall.brick[i2, j2] = 0` (which vaporizes a brick that has been hit) and insert the line

   ```
   Game.ships.append(Ship(0))
   ```

   below it.

   Test whether this works: every destroyed brick should now bring a new spaceship into the game.

   If everything works:

   ```
   git commit -a -m "moving stuff is in Game"
   git push
   ```

© Michael Lehn | CC BY-SA 4.0

Step 7: A completely different construction site

Test what happens when you **unplug the microcontroller while the game is running**.
The program then crashes.
According to the error message, the problem occurs in `controller.py`, more precisely in the method `readline()`.
It currently has this structure:

```
def readline():
    if Controller.ser is None:
        return None

    while Controller.ser.in_waiting > 0:
        # code to read the serial interface

    return None
```

What happens here?
If the microcontroller is removed while the game is running, `Controller.ser` is not `None`, but: as soon as you access the object (e.g., `Controller.ser.in_waiting`), it crashes.

The solution:
We put the while loop in which the object is used into a `try` block – and in the corresponding `except` block we catch the error by setting `Controller.ser` back to `None`.
So the structure is adapted like this:

```
def readline():
    if Controller.ser is None:
        return None

    try:
        while Controller.ser.in_waiting > 0:
            # code as before

    except:
        print("disconnected")
        Controller.ser = None
    return None
```

Task

Make this adaptation and test whether you can now remove the microcontroller while the game is running without the program crashing.

Note: By the way, if you have not put `ser` into a class Controller (i.e., your code contains `ser.in_waiting` instead of `Controller.ser.in_waiting`),
then the `except` block should of course simply contain `ser = None`.

Remember: When you reconnect the microcontroller, you have to run usb_check on the lab computers so that it is recognized again.

© Michael Lehn | CC BY-SA 4.0

Step 8: A few bombs to finish

The poor space invaders cannot defend themselves at all. In this step, we allow them to **drop bombs**. It won't really help them today, though – the bombs won't destroy our paddle (yet).
The main point here is to test **how easily a new feature can be built into the reorganized code**. And to show you **how to implement something like this quick and dirty, step by step**.

Task

1. At the very top of `ship.py`, import:

   ```
   import random
   from rocket import Rocket
   ```

   We need the module `random` for random numbers, and with `Rocket` we get our class for rockets. In the `move()` method of the Ship class, add these two lines:

   ```
   if random.randint(1, 100) == 1:
       Game.rockets.append(Rocket(self.x, self.y, 10))
   ```

   What happens here? In every frame, each spaceship has a 1% chance of dropping a "bomb". Okay, strictly speaking, it is not a bomb but a rocket – but those are details that can be changed later. 😄

   Test what happens! You should observe: the spaceships really do drop rockets downward (which, for now, still point in the wrong direction). But in this "attack", they currently destroy themselves.

2. **Why do the spaceships destroy themselves?** This happens because currently, for every object in Game.rockets, it is checked whether the distance to a spaceship is small enough for a contact.
   The solution: we create a new list Game.bombs in Game, so that bombs are managed separately from rockets.
   (a) Add a new, empty list `bombs` to `Game`, analogous to the existing lists `balls`, `rockets`, and `ships`.
   (b) Change the `draw` method of `Game` so that
   - all objects in `Game.balls`, `Game.rockets`, `Game.ships`, and now also in `Game.bombs` are moved and drawn.
   - objects that are no longer alive are removed from `Game.bombs`.

   (c) Change the bomb dropping in `ship.py` so that the rockets are appended to `Game.bombs`.

   Then test the game! The space invaders should now survive their bomb dropping.
3. Of course, it is still unattractive that rockets are currently simply dropped instead of bombs. No problem – by now you know how to solve something like this analogously!

   Use `cp rocket.py bomb.py` to create a copy of `rocket.py`.
   In `bomb.py`, you then make the following changes:
   (a) Name the class `Bomb`.
   (b) Use `data/bomb.png` as the image.
   (c) In the `move` method, make sure that `self.alive` is set to `False` if `self.y > Game.height`.

   And in `ship.py`, you now import the class `Bomb` instead of `Rocket`, and you also append a `Bomb` object instead of a `Rocket` object.
   Test whether bombs really fly now! If all is well, your code may go into the repository:

   ```
   git add bomb.py
   git commit -a -m "bomb blast"
   git push
   ```

© Michael Lehn | CC BY-SA 4.0